

LAPORAN TUGAS INDIVIDU 12

Klasifikasi Dua Kelas Dengan Logistic Regression

Nama : Fizar Erlansyah

NIM : 2024081041

Program Studi : Sistem Informasi

1. Latar Belakang

Dalam tugas ini, tema yang dipilih adalah klasifikasi sentimen positif dan negatif dari ulasan produk sederhana. Tujuannya adalah menggunakan Logistic Regression untuk memprediksi apakah sebuah teks ulasan bersifat positif atau negatif, berdasarkan fitur-fitur yang diekstrak dari teks tersebut.

Logistic Regression dipilih karena kemampuannya menghasilkan probabilitas untuk setiap kelas, yang lebih informatif dibandingkan prediksi keras (hard decision) saja. Metode ini juga mudah diinterpretasikan dan cocok untuk data berdimensi tinggi seperti representasi teks.

2. Dataset: 20 Data Teks dengan Label Dua Kelas

Dataset terdiri dari 20 teks ulasan produk yang diberi label secara manual:

- Label 1 (Positif): ulasan yang mengekspresikan kepuasan atau pujian — sebanyak 10 data.
- Label 0 (Negatif): ulasan yang mengekspresikan ketidakpuasan atau keluhan — sebanyak 10 data.

Tabel 1. Dataset Ulasan Produk

No	Teks Ulasan	Sentimen	Label
1	Produk sangat bagus dan berkualitas tinggi	Positif	1
2	Saya sangat puas dengan pembelian ini	Positif	1
3	Pengiriman cepat dan barang sesuai deskripsi	Positif	1
4	Kualitas produk sangat baik dan harga terjangkau	Positif	1
5	Seller responsif dan produk original	Positif	1
6	Barang diterima dalam kondisi sempurna	Positif	1
7	Sangat recommended, pasti beli lagi	Positif	1

8	Harga bersahabat tapi kualitas mewah	Positif	1
9	Pelayanan sangat ramah dan memuaskan	Positif	1
10	Fungsi berjalan normal dan tidak ada cacat	Positif	1
11	Produk rusak saat tiba, sangat mengecewakan	Negatif	0
12	Kualitas buruk dan tidak sesuai gambar	Negatif	0
13	Pengiriman lambat dan seller tidak responsif	Negatif	0
14	Barang tidak original, sangat kecewa	Negatif	0
15	Produk mudah rusak dan tidak tahan lama	Negatif	0
16	Bahan tipis dan jahitan tidak rapi	Negatif	0
17	Sangat menyesal beli di toko ini	Negatif	0
18	Respon penjual sangat lama dan tidak ramah	Negatif	0
19	Barang yang dikirim salah warna dan ukuran	Negatif	0
20	Packing asal-asalan sehingga kemasan penyok	Negatif	0

Total Data: 20 sampel | Positif: 10 | Negatif: 10

	Teks Ulasan	Sentimen	Target (0/1)
0	Produk sangat bagus dan berkualitas tinggi	● Positif	1
1	Saya sangat puas dengan pembelian ini	● Positif	1
2	Pengiriman cepat dan barang sesuai deskripsi	● Positif	1
3	Kualitas produk sangat baik dan harga terjangkau	● Positif	1
4	Seller responsif dan produk original	● Positif	1
5	Barang diterima dalam kondisi sempurna	● Positif	1
6	Sangat recommended, pasti beli lagi	● Positif	1
7	Harga bersahabat tapi kualitas mewah	● Positif	1
8	Pelayanan sangat ramah dan memuaskan	● Positif	1
9	Fungsi berjalan normal dan tidak ada cacat	● Positif	1
10	Produk rusak saat tiba, sangat mengecewakan	● Negatif	0
11	Kualitas buruk dan tidak sesuai gambar	● Negatif	0
12	Pengiriman lambat dan seller tidak responsif	● Negatif	0
13	Barang tidak original, sangat kecewa	● Negatif	0
14	Produk mudah rusak dan tidak tahan lama	● Negatif	0
15	Bahan tipis dan jahitan tidak rapi	● Negatif	0
16	Sangat menyesal beli di toko ini	● Negatif	0
17	Respon penjual sangat lama dan tidak ramah	● Negatif	0
18	Barang yang dikirim salah warna dan ukuran	● Negatif	0
19	Packing asal-asalan sehingga kemasan penyok	● Negatif	0

3. Penjelasan Fitur yang Relevan

Untuk klasifikasi teks, fitur yang digunakan adalah representasi numerik dari kata-kata dalam teks dengan metode TF-IDF (Term Frequency–Inverse Document Frequency).

3.1 Mengapa TF-IDF?

Tabel 2. Komponen Fitur TF-IDF

Komponen	Penjelasan
TF (Term Frequency)	Mengukur seberapa sering sebuah kata muncul dalam satu dokumen. Kata yang sering muncul dianggap penting bagi dokumen tersebut.
IDF (Inverse Document Frequency)	Mengurangi bobot kata-kata umum yang muncul di banyak dokumen (seperti "dan", "yang"). Kata yang jarang mendapat bobot lebih tinggi.
TF-IDF (Gabungan)	Kata yang sering muncul di satu dokumen tetapi jarang di dokumen lain dianggap sebagai fitur yang paling diskriminatif untuk klasifikasi.

3.2 Kata Kunci per Kelas

- Positif: bagus, puas, cepat, berkualitas, sempurna, original, recommended
- Negatif: rusak, buruk, lambat, kecewa, mengecewakan, tidak

Fitur TF-IDF secara otomatis menangkap pola kata-kata di atas dan memberikan bobot yang sesuai untuk membantu model membedakan kedua kelas.

4. Mengapa Logistic Regression Cocok untuk Kasus Ini?

Logistic Regression adalah pilihan yang tepat untuk kasus klasifikasi sentimen biner (dua kelas) karena beberapa alasan:

- **Output Probabilitas:** Model menghasilkan nilai probabilitas antara 0 dan 1 menggunakan fungsi Sigmoid, sehingga kita tidak hanya tahu kelas prediksinya, tetapi juga seberapa yakin model tersebut.
- **Efisien untuk Data Teks:** Cocok digunakan bersama TF-IDF karena dapat menangani data berdimensi tinggi dengan baik.

- **Mudah Diinterpretasikan:** Bobot (koefisien) tiap fitur bisa dilihat, sehingga kita bisa mengetahui kata mana yang paling berkontribusi terhadap klasifikasi positif atau negatif.
- **Dirancang untuk Klasifikasi Biner:** Secara matematis, Logistic Regression memang dirancang khusus untuk memecahkan masalah dua kelas.

4.1 Rumus Sigmoid

Model menggunakan fungsi Sigmoid untuk mengubah hasil linear menjadi nilai probabilitas:

$$P(y=1 \mid \mathbf{x}) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)})$$

Aturan keputusan: Jika $P \geq 0.5 \rightarrow$ prediksi Positif; jika $P < 0.5 \rightarrow$ prediksi Negatif.

5. Implementasi Logistic Regression dengan Python

5.1 Ekstraksi Fitur dan Pembagian Data

Langkah pertama adalah mengubah teks menjadi representasi numerik menggunakan TfidfVectorizer dari scikit-learn, lalu membagi data menjadi data latih (75%) dan data uji (25%).

```
# =====
# Ekstraksi Fitur & Pembagian Data
# =====
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['teks'])
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```



Model Logistic Regression Berhasil Dilatih!

- **Jumlah Fitur (Kata Unik):** 74 kata
- **Data Training:** 15 sampel
- **Data Testing:** 5 sampel

5.2 Pelatihan Model

Model Logistic Regression dilatih menggunakan data latih. Setelah pelatihan, model siap digunakan untuk memprediksi data baru.

```
# =====  
# Pelatihan Model Logistic Regression  
# =====  
model = LogisticRegression(max_iter=1000, random_state=42)  
model.fit(X_train, y_train)  
  
display(Markdown(f"""  
### ✅ Model Logistic Regression Berhasil Dilatih!  
- **Jumlah Fitur (Kata Unik):** {X.shape[1]} kata  
- **Data Training:** {X_train.shape[0]} sampel  
- **Data Testing:** {X_test.shape[0]} sampel  
"""))
```



Model Logistic Regression Berhasil Dilatih!

5.3 Prediksi dan Evaluasi

Model diuji pada data testing. Tabel berikut menunjukkan hasil prediksi beserta tingkat keyakinan (probabilitas) untuk setiap sampel:

```
# =====  
# STEP 5: Prediksi dan Evaluasi  
# =====  
y_pred = model.predict(X_test)  
y_prob = model.predict_proba(X_test)  
  
test_indices = y_test.index.tolist()  
teks_test = df.loc[test_indices, 'teks'].tolist()  
  
# Buat tabel hasil prediksi  
hasil = pd.DataFrame({  
    'Teks Ulasan'      : teks_test,  
    'Label Asli'       : ['🟢 Positif' if v == 1 else '🔴 Negatif' for v in y_test],  
    'Prediksi'        : ['🟢 Positif' if v == 1 else '🔴 Negatif' for v in y_pred],  
    'P(Negatif)'       : [f"{p[0]:.1%}" for p in y_prob],  
    'P(Positif)'       : [f"{p[1]:.1%}" for p in y_prob],  
    'Status'           : ['✅ Benar' if a == p else '❌ Salah' for a, p in zip(y_test, y_pred)]  
})  
  
display(Markdown("### 📊 Hasil Prediksi pada Data Testing"))  
display(hasil)  
  
# Tampilkan Akurasi  
akurasi = accuracy_score(y_test, y_pred)  
display(Markdown(f"""🏆 Akurasi Model: {akurasi:.2%}"""))
```

🔗 Hasil Prediksi pada Data Testing						
	Teks Ulasan	Label Asli	Prediksi	P(Negatif)	P(Positif)	Status
0	Produk sangat bagus dan berkualitas tinggi	🟢 Positif	🔴 Negatif	52.7%	47.3%	❌ Salah
1	Respon penjual sangat lama dan tidak ramah	🔴 Negatif	🔴 Negatif	57.7%	42.3%	✅ Benar
2	Bahan tipis dan jahitan tidak rapi	🔴 Negatif	🔴 Negatif	55.5%	44.5%	✅ Benar
3	Saya sangat puas dengan pembelian ini	🟢 Positif	🔴 Negatif	55.3%	44.7%	❌ Salah
4	Pelayanan sangat ramah dan memuaskan	🟢 Positif	🔴 Negatif	53.5%	46.5%	❌ Salah
👤 Akurasi Model: 40.00%						

5.4 Visualisasi Hasil

Dua visualisasi dihasilkan dari proses ini: grafik probabilitas prediksi untuk setiap data testing, dan confusion matrix yang menunjukkan perbandingan antara label asli dan hasil prediksi model.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# =====
# STEP 6: Visualisasi – Probabilitas Prediksi per Data Test
# =====

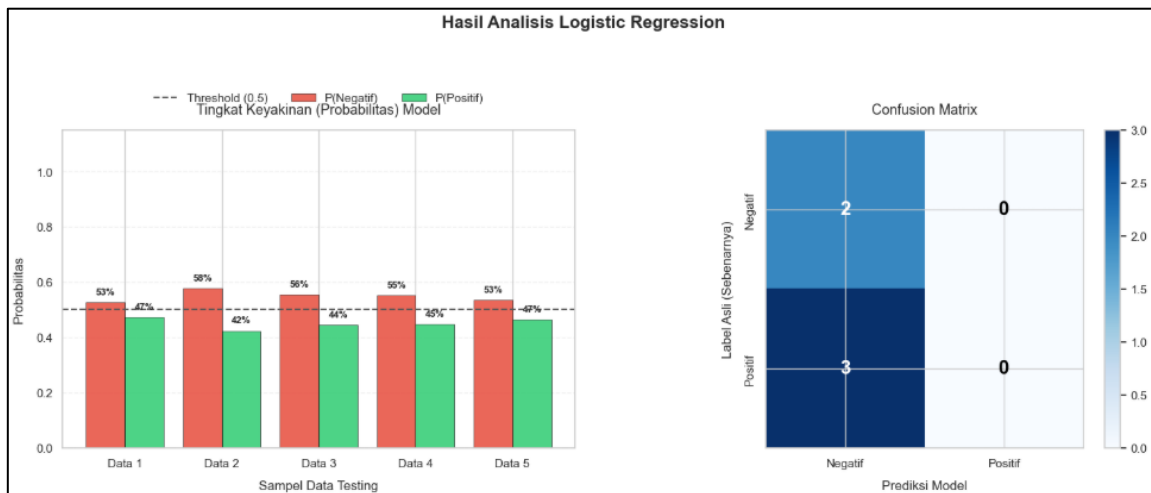
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
fig.suptitle('Hasil Analisis Logistic Regression', fontsize=16, fontweight='bold', y=1.05)

# --- Plot 1: Probabilitas tiap data test ---
ax1 = axes[0]
x_pos = np.arange(len(teks_test))
bars_neg = ax1.bar(x_pos - 0.2, [p[0] for p in y_prob], 0.4,
                  label='P(Negatif)', color='#e74c3c', alpha=0.85, edgecolor='black', linewidth=0.5)
bars_pos = ax1.bar(x_pos + 0.2, [p[1] for p in y_prob], 0.4,
                  label='P(Positif)', color='#2ecc71', alpha=0.85, edgecolor='black', linewidth=0.5)
ax1.axhline(y=0.5, color='#555555', linestyle='--', linewidth=1.5, label='Threshold (0.5)')
ax1.set_xlabel('Sampel Data Testing', fontsize=12, labelpad=10)
ax1.set_ylabel('Probabilitas', fontsize=12, labelpad=10)
ax1.set_title('Tingkat Keyakinan (Probabilitas) Model', fontsize=13, pad=15)
ax1.set_xticks(x_pos)
ax1.set_xticklabels([f'Data {i+1}' for i in range(len(teks_test))], fontsize=11)
ax1.set_ylim(0, 1.15)
ax1.legend(loc='upper center', bbox_to_anchor=(0.5, 1.15), ncol=3, frameon=False)
ax1.grid(axis='y', alpha=0.3, linestyle='--')

# Tambahkan nilai persentase di atas bar
for bar in bars_neg:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
            f'{bar.get_height():.0%}', ha='center', va='bottom', fontsize=9, fontweight='bold')
for bar in bars_pos:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
            f'{bar.get_height():.0%}', ha='center', va='bottom', fontsize=9, fontweight='bold')
```

```
# --- Plot 2: Confusion Matrix ---
ax2 = axes[1]
cm = confusion_matrix(y_test, y_pred)
im = ax2.imshow(cm, interpolation='nearest', cmap='Blues')
plt.colorbar(im, ax=ax2, fraction=0.046, pad=0.04)
classes = ['Negatif', 'Positif']
tick_marks = np.arange(len(classes))
ax2.set_xticks(tick_marks)
ax2.set_xticklabels(classes, fontsize=11)
ax2.set_yticks(tick_marks)
ax2.set_yticklabels(classes, fontsize=11, rotation=90, va='center')
ax2.set_xlabel('Prediksi Model', fontsize=12, labelpad=10)
ax2.set_ylabel('Label Asli (Sebenarnya)', fontsize=12, labelpad=10)
ax2.set_title('Confusion Matrix', fontsize=13, pad=15)
thresh = cm.max() / 2
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax2.text(j, i, format(cm[i, j], 'd'),
                ha='center', va='center',
                color='white' if cm[i, j] > thresh else 'black',
                fontsize=18, fontweight='bold')

plt.tight_layout()
plt.savefig('hasil_prediksi.png', dpi=150, bbox_inches='tight')
plt.show()
display(Markdown(f"📄 Grafik visualisasi telah disimpan sebagai **hasil_prediksi.png** di folder yang sama.."))
```



5.5 Kata-Kata Paling Berpengaruh

Logistic Regression mempelajari kata mana yang paling berkontribusi pada prediksi positif atau negatif berdasarkan nilai koefisien yang dipelajari model.

```
# =====
# STEP 7: Top Kata yang Paling Berpengaruh
# =====
feature_names = vectorizer.get_feature_names_out()
coef = model.coef_[0]

# Urutkan dari terbesar (positif) ke terkecil (negatif)
sorted_idx = np.argsort(coef)

top_negatif = [(feature_names[i], coef[i]) for i in sorted_idx[:5]]
top_positif = [(feature_names[i], coef[i]) for i in sorted_idx[-5:][::-1]]

df_negatif = pd.DataFrame(top_negatif, columns=['Kata Pendorong', 'Negatif', 'Bobot'])
df_positif = pd.DataFrame(top_positif, columns=['Kata Pendorong', 'Positif', 'Bobot'])

display(Markdown("### 💡 Analisis Kata: Kata-Kata Paling Berpengaruh"))
display(Markdown("Logistic Regression mempelajari kata mana yang menandakan sentimen positif atau negatif berdasarkan bobot koefisiennya."))

html_str = f"""
<div style="display: flex; gap: 20px;">
  <div>{df_negatif.to_html(index=False)}</div>
  <div>{df_positif.to_html(index=False)}</div>
</div>
"""
display(HTML(html_str))
```

💡 Analisis Kata: Kata-Kata Paling Berpengaruh

Logistic Regression mempelajari kata mana yang menandakan sentimen positif atau negatif berdasarkan bobot koefisiennya.

Kata Pendorong	Negatif	Bobot	Kata Pendorong	Positif	Bobot
	tidak	-0.376649		harga	0.352761
	rusak	-0.308116		deskripsi	0.227232
	kecewa	-0.236296		cepat	0.227232
	lambat	-0.218833		recommended	0.224106
	gambar	-0.206467		pasti	0.224106

6. Alur Kerja Logistic Regression (Konseptual)

Berikut adalah alur lengkap dari proses yang telah dijalankan dari input teks mentah hingga output keputusan kelas:

1. Input Teks Mentah

Contoh: "Produk sangat bagus dan berkualitas tinggi"

↓

2. Ekstraksi Fitur (TF-IDF Vectorizer)

Teks → Vektor numerik [0.0, 0.45, 0.0, 0.38, ...]

↓

3. Model Logistic Regression

$$z = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_n \cdot x_n$$

$$P(\text{Positif}) = 1 / (1 + e^{(-z)}) \leftarrow \text{Fungsi Sigmoid}$$

↓

4. Output Probabilitas

$$P(\text{Positif}) = 0.92 \rightarrow P(\text{Negatif}) = 0.08$$

↓

5. Keputusan Kelas

$$P \geq 0.5 \rightarrow \text{Prediksi} = \text{POSITIF}$$

$$P < 0.5 \rightarrow \text{Prediksi} = \text{NEGATIF}$$

7. Probabilitas vs Keputusan Keras

Salah satu keunggulan utama Logistic Regression adalah kemampuannya menghasilkan probabilitas, bukan hanya keputusan keras (0 atau 1). Tabel berikut merangkum perbedaan keduanya:

Tabel 3. Perbandingan Probabilitas vs Keputusan Keras

Aspek	Keputusan Keras	Probabilitas
Output	0 atau 1 saja	Nilai antara 0.0 – 1.0
Informasi	Minimal	Lebih kaya dan informatif
Contoh Output	"Positif"	"Positif dengan keyakinan 92%"
Penggunaan	Klasifikasi akhir saja	Ranking, threshold fleksibel
Kelebihan Utama	Sederhana dan mudah dipahami	Batas keputusan dapat disesuaikan

8. Kesimpulan

Dalam tugas ini, Logistic Regression berhasil diimplementasikan untuk mengklasifikasikan ulasan produk ke dalam dua kelas: Positif dan Negatif. Model bekerja dengan mengubah teks mentah menjadi representasi numerik menggunakan TF-IDF, kemudian mempelajari bobot (koefisien) untuk setiap kata yang paling membedakan kedua kelas — misalnya kata "bagus", "puas", dan "sempurna" cenderung mendorong prediksi positif, sedangkan kata "rusak", "kecewa", dan "buruk" mendorong prediksi negatif.

Kelebihan utama Logistic Regression dibanding metode keputusan keras adalah kemampuannya menghasilkan nilai probabilitas untuk setiap prediksi. Dengan demikian, kita tidak hanya mengetahui kelas yang diprediksi, tetapi juga tingkat keyakinan model — misalnya "Positif dengan probabilitas 94%" jauh lebih informatif daripada sekadar "Positif", dan memungkinkan kita untuk menyesuaikan ambang batas keputusan sesuai kebutuhan bisnis.

9. Tambahan (Ringkasan Materi Pertemuan 12)

Materi tersebut membahas tentang *logistic regression*, sebuah algoritma *machine learning* yang digunakan untuk melakukan klasifikasi, khususnya untuk klasifikasi dua kelas (*two-class classification*). Meskipun memiliki kata "*regression*" di namanya, algoritma ini tidak digunakan untuk memprediksi nilai kontinu melainkan untuk menghasilkan kategori atau keputusan kelas. Sebagai model yang termasuk ke dalam kelompok *linear classifier*, *logistic regression* bertindak sebagai *probabilistic-boundary classifier* yang bekerja secara lebih informatif dibandingkan Perceptron. Hal ini dikarenakan model tidak langsung memberikan keputusan akhir secara kaku, melainkan menghitung skor linear dari fitur-fitur input yang ada menggunakan bobot dan bias, lalu mengonversinya menjadi nilai probabilitas di antara 0 dan 1 dengan memanfaatkan fungsi sigmoid. Setelah nilai peluang didapatkan, barulah model menentukan keputusan kelas berdasarkan batas keputusan (*decision boundary*) yang biasanya menggunakan ambang batas (*threshold*) 0.5.

Pada saat proses pelatihan (*training*), model akan menyesuaikan parameter bobot secara bertahap untuk mencari kombinasi terbaik agar hasil prediksi semaksimal mungkin sesuai dengan data sebenarnya. Proses mencari parameter ideal ini secara akademik dikenal sebagai memaksimalkan fungsi *likelihood*, yang secara praktis diartikan sebagai upaya untuk meminimalkan *error* atau *loss* agar tingkat kebenaran prediksi semakin tinggi. Penerapan dari algoritma ini sangat cocok untuk berbagai studi kasus dunia nyata yang memiliki pilihan biner, seperti deteksi komentar *toxic* di media sosial, penyaringan email *spam*, analisis sentimen

ulasan positif/negatif, maupun identifikasi konten *clickbait*. Terakhir, untuk mengukur kinerja dan performa dari model yang telah dilatih, evaluasi dilakukan menggunakan metrik standar seperti *confusion matrix*, *accuracy*, *precision*, *recall*, dan *F1-score*.